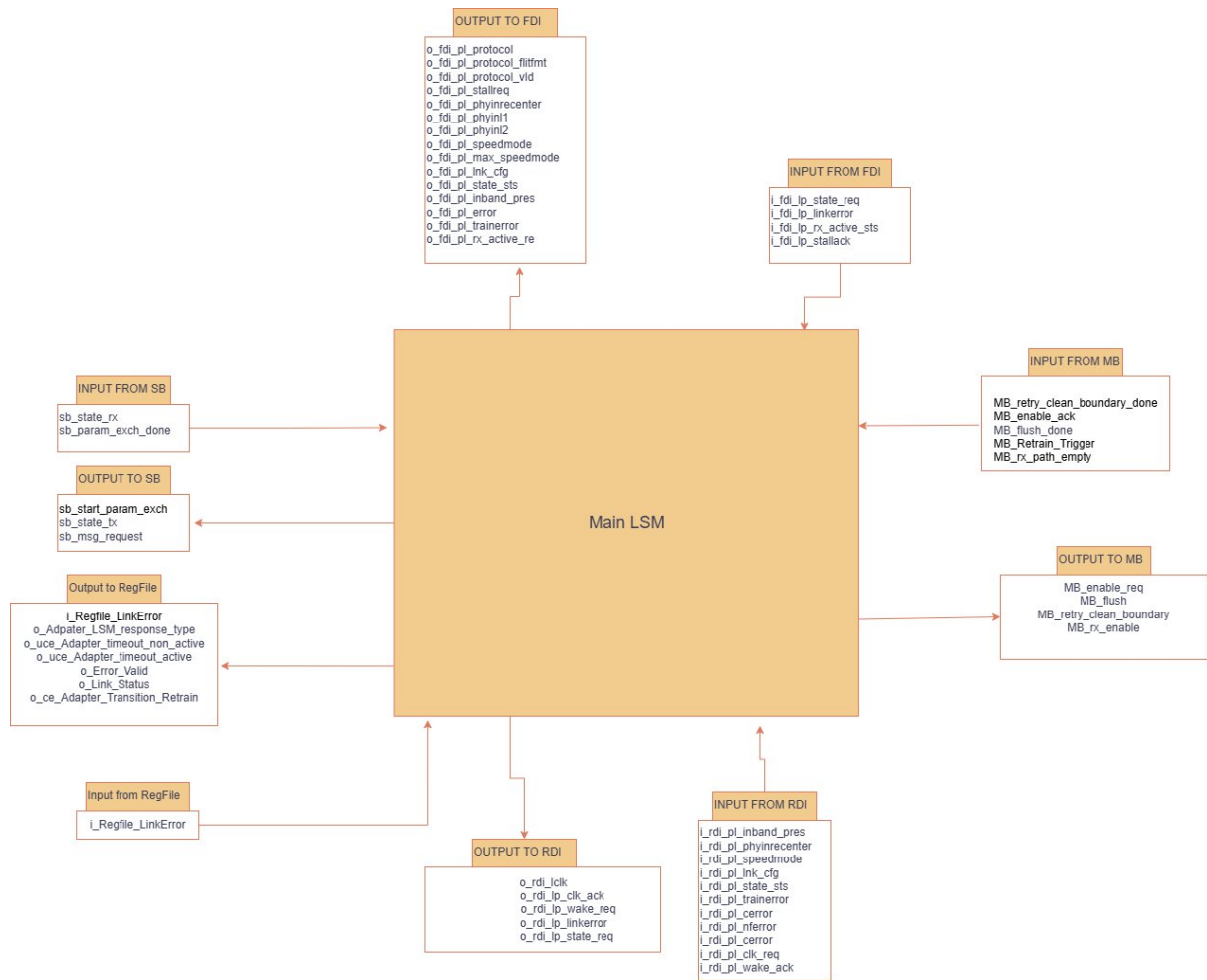


Main LSM



Functionality:

FDI and RDI:

The Adapter LSM block is responsible for handling the RDI and FDI state machine hierarchy. Implementing all states for RDI and FDI according to specification.

Mainband:

It controls the MB block for each state (when to start and when to halt communication) depending on the ALSM specification (Total Halt, Active Exit, Retrain Entry, PM Entry, Active Entry, LinkDown and LinkError).

Sideband:

The ALSM uses the Sideband block to communicate with the Remote link partner for State Transition messages and parameter exchange initialization.

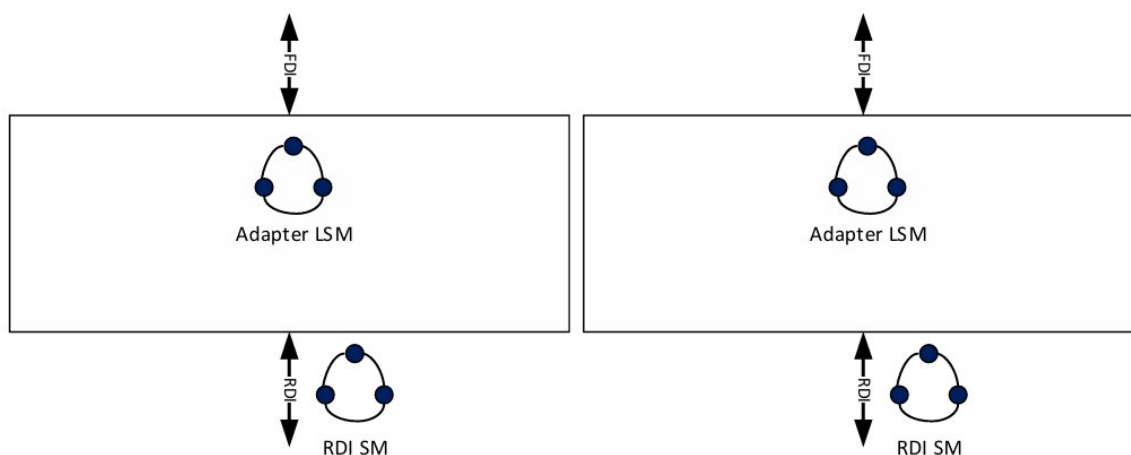
Register File:

The Register File is used as a global error handler and is therefore given all possible errors that the ALSM can generate. The ALSM also uses the Uncorrectable Error Signal given out by the Register File to transition To/From LinkError State.

Purpose:

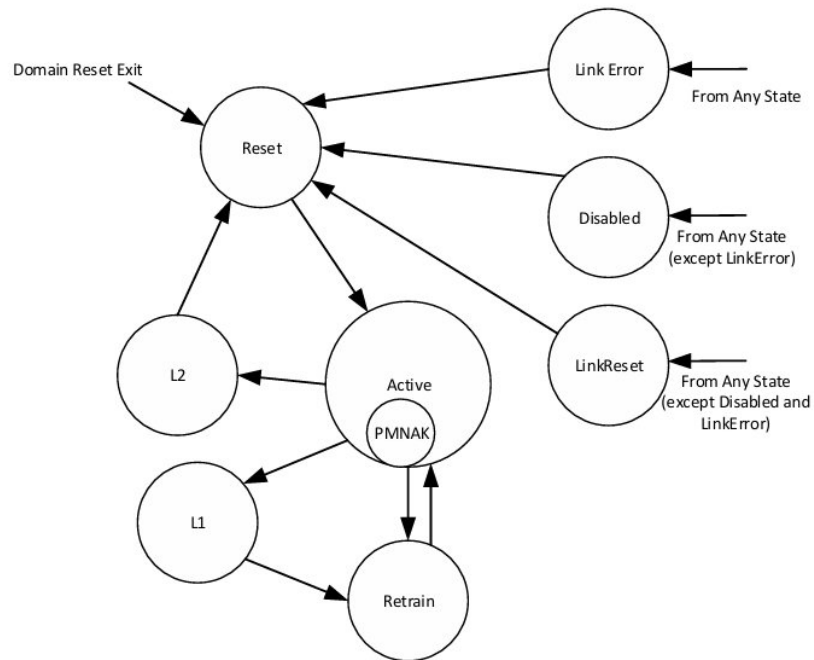
The Adapter Link State Machine (ALSM) is a foundational component of the Die-to-Die (D2D) Adapter responsible for coordinating high-level link management and synchronization between the local and remote dies. It serves as the primary mechanism for state negotiation, using dedicated sideband messages to ensure that both link partners reach a common understanding of the link's operational status.

The ALSM acts as a critical abstraction layer that consolidates the diverse states of the underlying hardware into a unified status for the upper Protocol Layer. It effectively shields the Protocol Layer from the Physical Layer's sub-states (which are themselves abstracted via the RDI state machine) while providing a consistent interface. This consolidation allows the system to manage transitions between complex states like Power Management (L1/L2) or Retrain by ensuring that all internal buffers, such as the Retry buffer, are properly flushed/emptied and aligned at clean boundaries before the state change is finalized across the dies. Each die has its own FDI and RDI state machine like so:

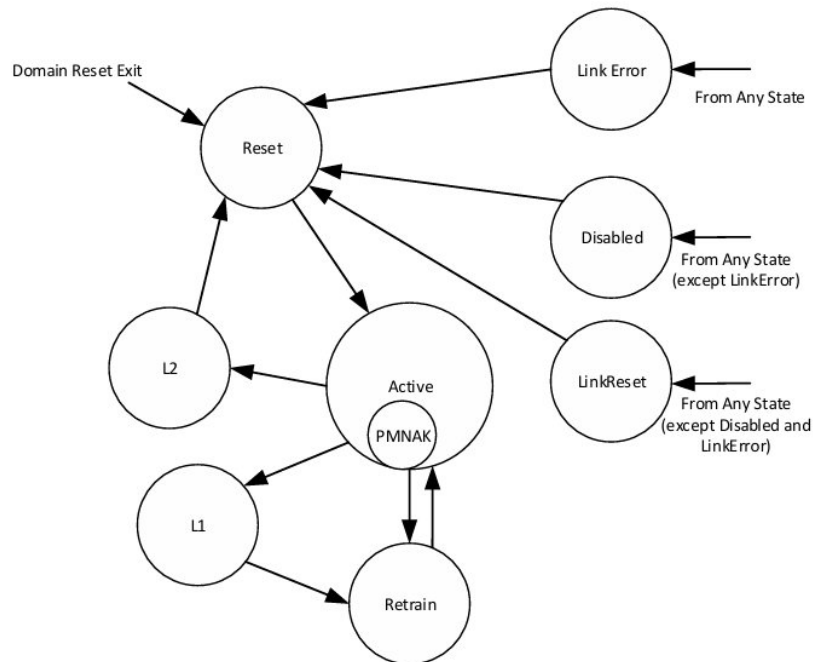


Each State machine consists of the following states as per spec:

RDI State Machine



FDI State Machine



The FDI and RDI state machines are the same although they represent different layers (the Protocol and PHY layer respectively). State transitions are to be negotiated on the FDI and RDI on the same die and on the Sideband for remote die state transition.

IOs:

Port	Direction	Width	Description
i_rdi_pl_inband_pres	Input	1	RDI can only request active when this signal is high
i_rdi_pl_phyinrecenter	Input	1	RDI indication that phy is in train or retrain
i_rdi_pl_speedmode	Input	3	To be registered and given to the FDI
i_rdi_pl_lnk_cfg	Input	3	To be registered and given to the FDI
i_rdi_pl_state_sts	Input	4	Reflects the state of the RDI
i_rdi_pl_clk_req	Input	1	Phy starts the four way handshake of rdi ungating
i_rdi_pl_wake_ack	Input	1	Adapter starts the four way handshake of rdi ungating
o_rdi_lclk	Input	1	Clock at which the rdi interface operates
o_rdi_lp_clk_ack	Output	1	Registered rdi pl clk req
o_rdi_lp_wake_req	Output	1	Adapter starts the four way handshake of rdi ungating (tied to 1b)
o_rdi_lp_linkerror	Output	1	Indication to Phy error occurred
o_rdi_lp_state_req	Output	4	RDI state request to PHY
i_fdi_lp_state_req	Input	4	FDI state request to Adapter
i_fdi_lp_linkerror	Input	1	Indication to Phy error occurred
i_fdi_lp_rx_active_sts	Input	1	Reflects the state of the Rx path of the Protocol Layer
i_fdi_lp_stall_ack	Input	1	Indicates that Protocol has stopped transmitting on a flit boundary
o_fdi_pl_stallreq	Output	1	Stall signal on FDI when entering PM
o_fdi_pl_phyinrecenter	Output	1	(FDI in retrain OR rdi pl phyinrecenter)

			and lp_clk_ack
o_fdi_pl_phyinl1	Output	1	Indication when phy in L1 PM
o_fdi_pl_phyinl2	Output	1	Indication when phy in L2 PM
o_fdi_pl_speedmode	Output	3	Registered fdi_pl_speedmode
o_fdi_pl_max_speedmode	Output	1	Registered fdi_pl_max_speedmode
o_fdi_pl_lnk_cfg	Output	3	Registered o_fdi_pl_lnk_cfg
o_fdi_pl_state_sts	Output	4	Indication of FDI state
o_fdi_pl_inband_pres	Output	1	FDI can only request Active if this signal is high and clk_req handshake is done
o_fdi_pl_rx_active_req	Output	1	Requests that protocol opens its Rx path
i_sb_state_rx	Input	4	SB message to be sent
i_sb_param_exch_done	Input	1	SB indication that parameter exchange is done
o_sb_start_param_exch	Output	1	ALSM trigger to start parameter exchange
o_sb_msg_request	Output	1	ALSM trigger to send SB message
o_sb_state_tx	Output	4	ALSM state request over the sideband
i_MB_retry_clean_boundary_done	Input	1	MB indication that a clean boundary is reached and pl_trdy is asserted
i_MB_flush_done	Input	1	Indication that the Tx retry buffer is empty
i_MB_Retrain_Trigger	Input	1	MB trigger for the ALSM to enter Retrain
i_MB_rx_path_empty	Input	1	Indication that the rx path of the MB is empty
o_MB_flush	Output	1	ALSM trigger for the MB to deassert trdy and flush the retry buffer
o_MB_retry_clean_boundary	Output	1	ALSM trigger for the MB to deassert trdy and reach a flit boundary and stop transmitting
o_MB_tx_enable	Output	1	ALSM for the MB to enable the TX path
o_MB_rx_enable	Output	1	ALSM for the MB to enable the RX path
i_Regfile_LinkError	Input	1	Global Uncorrectable Error Signal
o_Adapter_LSM_response_type	Output	3	Error code of the

			ALSM transition
o_uce_Adapter_timeout_non_active	Output	1	Timeout not in active state
o_uce_Adapter_timeout_active	Output	1	Timeout in non-active state
o_Error_Valid	Output	1	Error Valid trigger
o_Link_Status	Output	1	UP/DOWN status
o_ce_Adapter_Transition_Retrain	Output	1	Indicator when ALSM goes to retrain

Implementation:

Link Bringup:

Link Bringup is the process of taking a link from Reset to Active state. Bringup process must occur on the FDI and RDI state machines like so for the Link to start Active like so:

Figure 10-7. Example flow of Link bring up on RDI

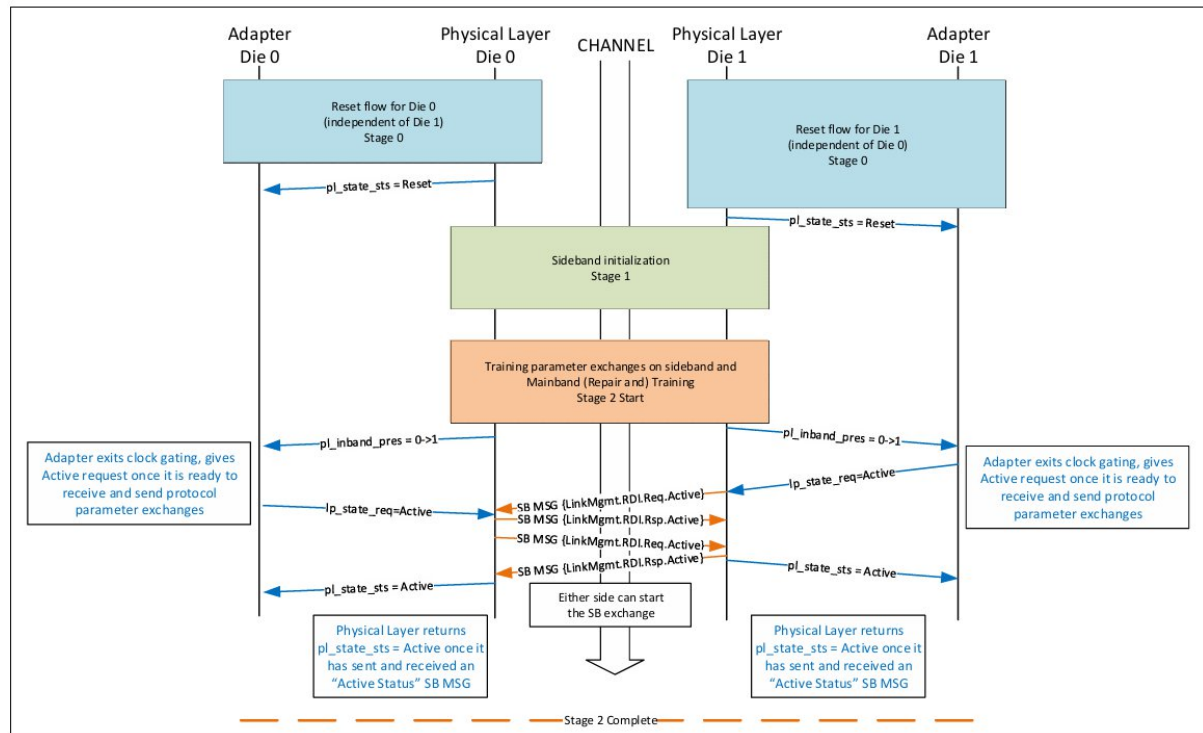
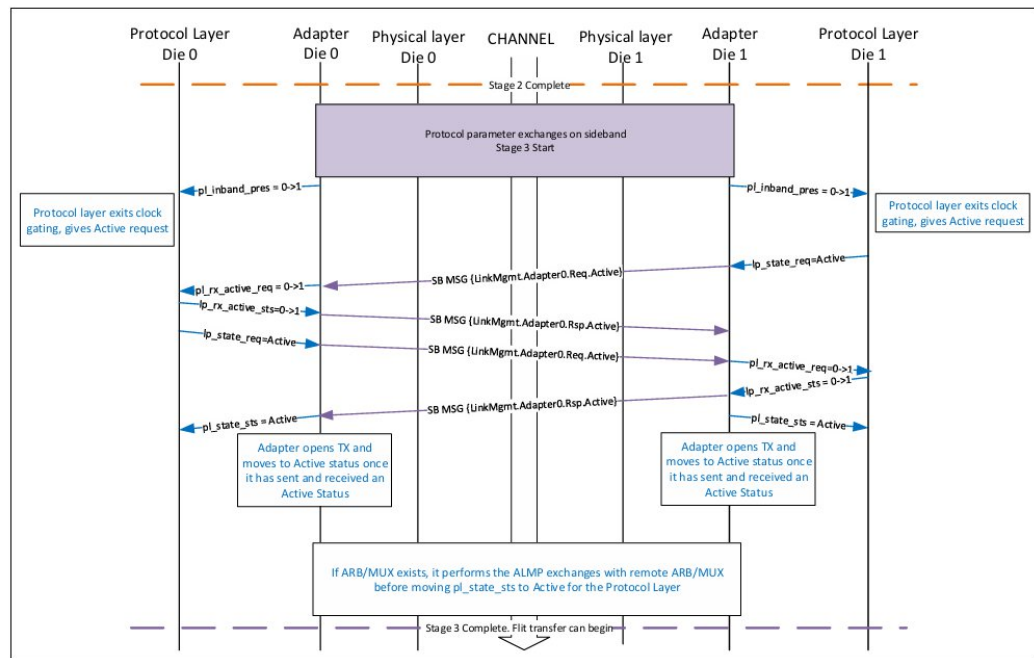
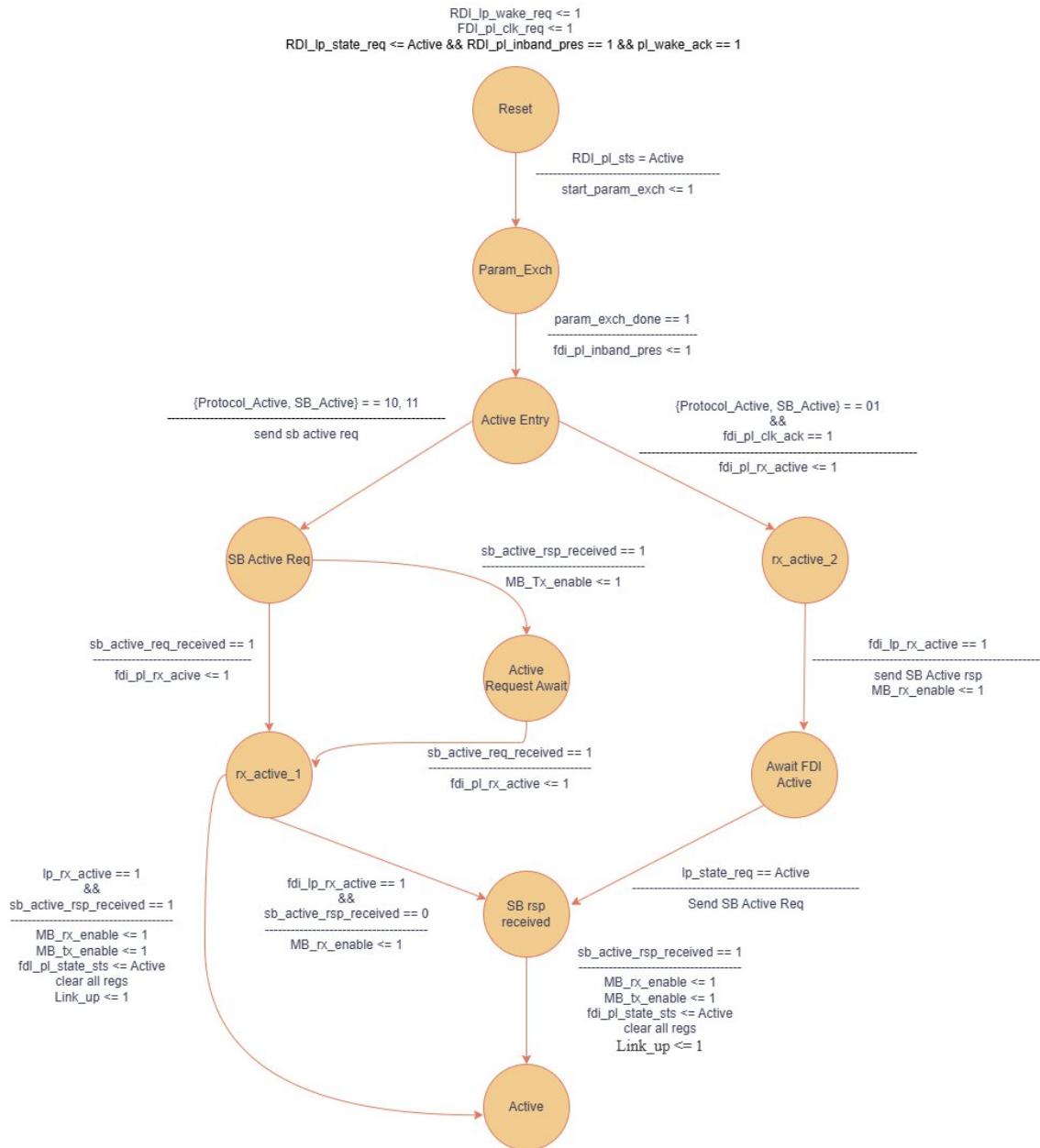


Figure 10-20. FDI Bring up flow



Note that there is a separate RDI and FDI bringup flows where the RDI flow is negotiated by the PHY and reflected on the RDI interface and the FDI flow is negotiated by the Adapter Sideband messages.

The implemented flow in the ALSM is like the following:

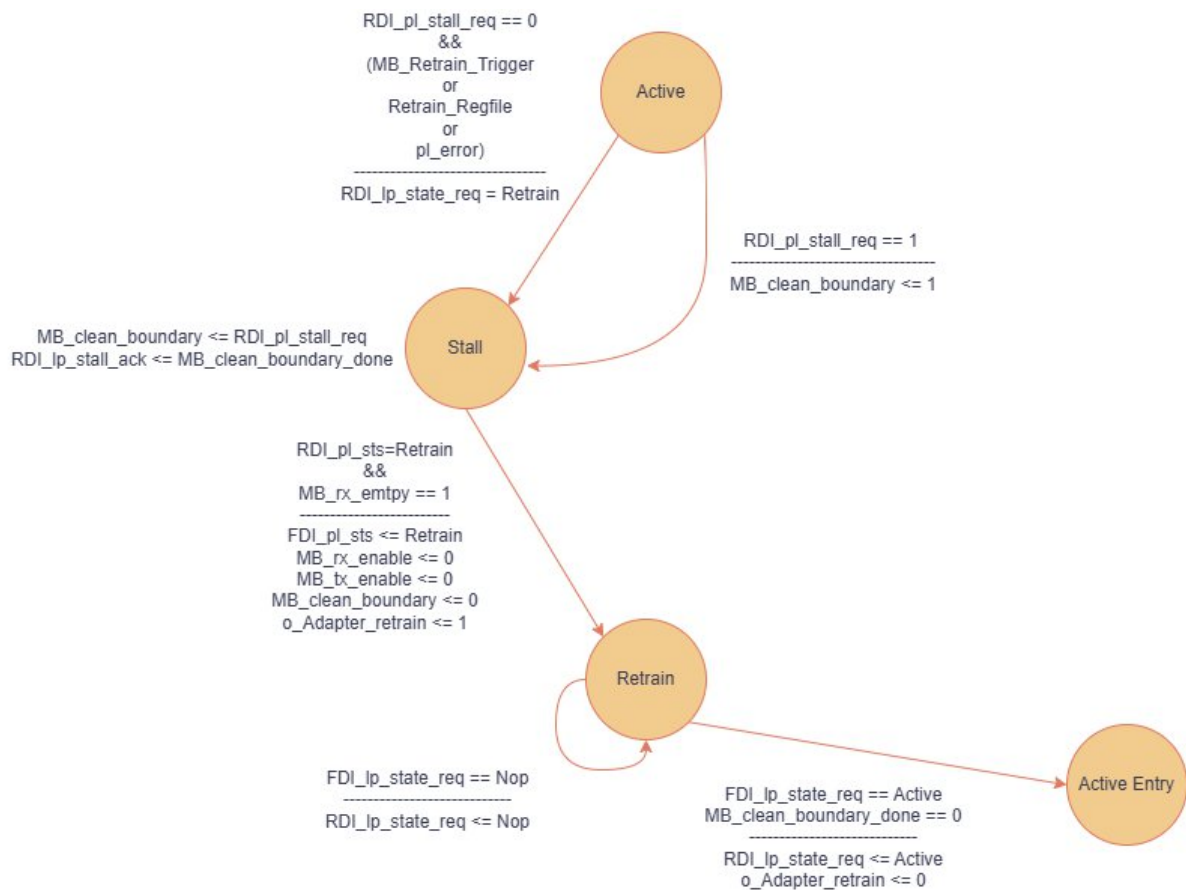


Our ALSM implementation consolidates the RDI and FDI bringup flows and ensures proper activation of Mainband Tx and Rx paths after parameter exchange while accounting for different scenarios concerning the order at which the Sideband Messages arrive.

Retrain:

Retrain is sometimes necessary when operating the Mainband. Cases which they are are when there are many CRC errors or many flits are replayed due to errors. Retrain can be started from the remote link partner or from the Mainband.

In order to train/retrain the link the Mainband must be off. This is achieved by communicating to the RDI about turning off the Tx path on a flit boundary (not to send partial flits) and then enter Retrain once the Rx path is empty. Retrain State machine is implemented like the following:



Retrain can only be entered by the stall_req/ack handshake offered by the RDI, this ensures that the Tx path is empty. To go back to Active the Protocol Layer must first request it on the lp_state_req signal on the FDI and then the state machine can go to active but NOT redoing parameter exchange.

LinkError:

The UCIE offers a state for when Uncorrectable errors occur in a Link. Sources of Errors can be invalid opcodes on the Sideband, framing errors from the PHY or the Mainband, Parity Errors from the Sideband, overflow Errors from the Sideband or Mainband, and many other Link-Defined cases.

LinkError can be requested from upper layers, but entered from the bottom layers first and then LinkError entry is propagated upwards. To enter LinkError state, the Mainband must flush the Tx buffer before being disabled. An example of LinkError from the spec is:

The diagram illustrates the sequence of events and state transitions during a Link Down Event and Reset process. The participants involved are the DP Protocol Layer (Die 0), DP Adapter (Die 0), DP Physical layer (Die 0), CHANNEL, UP Physical layer (Die 1), UP Adapter (Die 1), and UP Protocol Layer (Die 1).

Initial State: The DP Protocol Layer (Die 0) has `pl_state_sts = Active`. The DP Adapter (Die 0) has `lp_linkerror=1`. The DP Physical layer (Die 0) has `lp_linkerror=1`. The UP Physical layer (Die 1) has `pl_state_sts = LinkError`. The UP Adapter (Die 1) has `pl_state_sts = LinkError`. The UP Protocol Layer (Die 1) has `pl_state_sts = LinkError`.

Link Down Event: The DP Protocol Layer (Die 0) detects a Link Down Event and asserts the `LinkError` signal. This triggers the DP Adapter (Die 0) to assert `lp_linkerror=1`, which in turn asserts `lp_linkerror=1` to the DP Physical layer (Die 0). The DP Physical layer (Die 0) then asserts `SB MSG (LinkAgmt_RDLReq.LinkError)` to the CHANNEL. The CHANNEL asserts `SB MSG (LinkAgmt_RDLReq.LinkError)` to the UP Physical layer (Die 1), which asserts `pl_state_sts = LinkError` to the UP Adapter (Die 1). The UP Adapter (Die 1) then asserts `pl_state_sts = LinkError` to the UP Protocol Layer (Die 1).

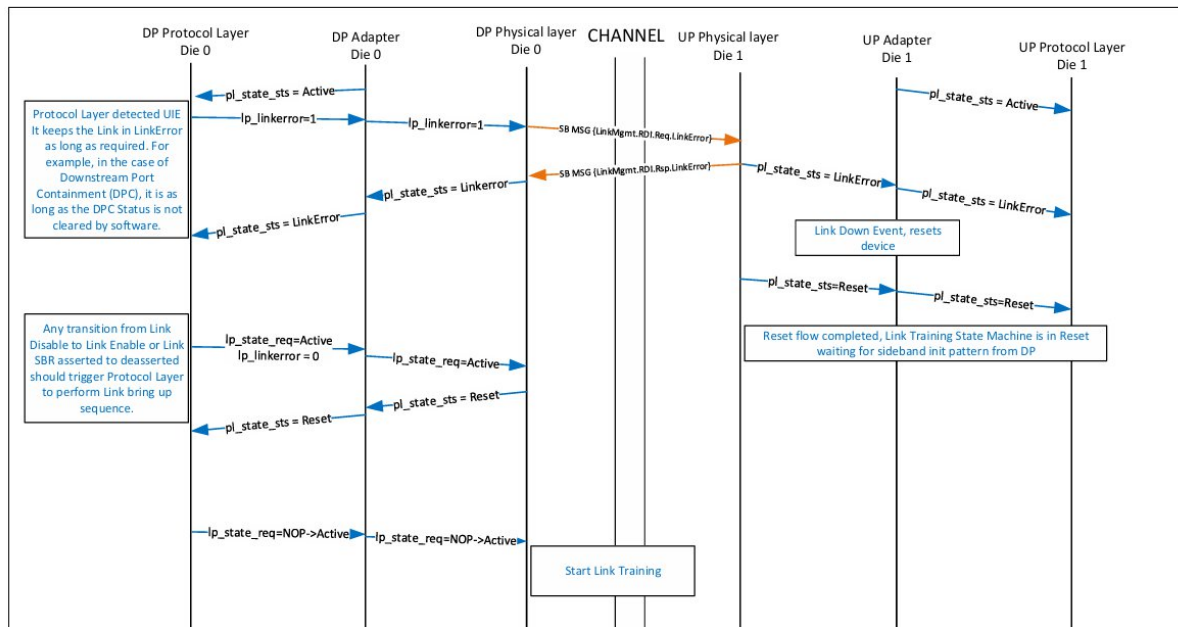
Reset Process: The DP Protocol Layer (Die 0) asserts `pl_state_sts = LinkError` to the DP Adapter (Die 0). The DP Adapter (Die 0) asserts `pl_state_sts = LinkError` to the DP Physical layer (Die 0). The DP Physical layer (Die 0) asserts `pl_state_sts = LinkError` to the CHANNEL. The CHANNEL asserts `pl_state_sts = LinkError` to the UP Physical layer (Die 1), which asserts `pl_state_sts = LinkError` to the UP Adapter (Die 1). The UP Adapter (Die 1) then asserts `pl_state_sts = LinkError` to the UP Protocol Layer (Die 1).

Link Down Event, resets device: The UP Adapter (Die 1) asserts `pl_state_sts = Reset` to the UP Protocol Layer (Die 1). The UP Protocol Layer (Die 1) asserts `pl_state_sts = Reset` to the UP Adapter (Die 1).

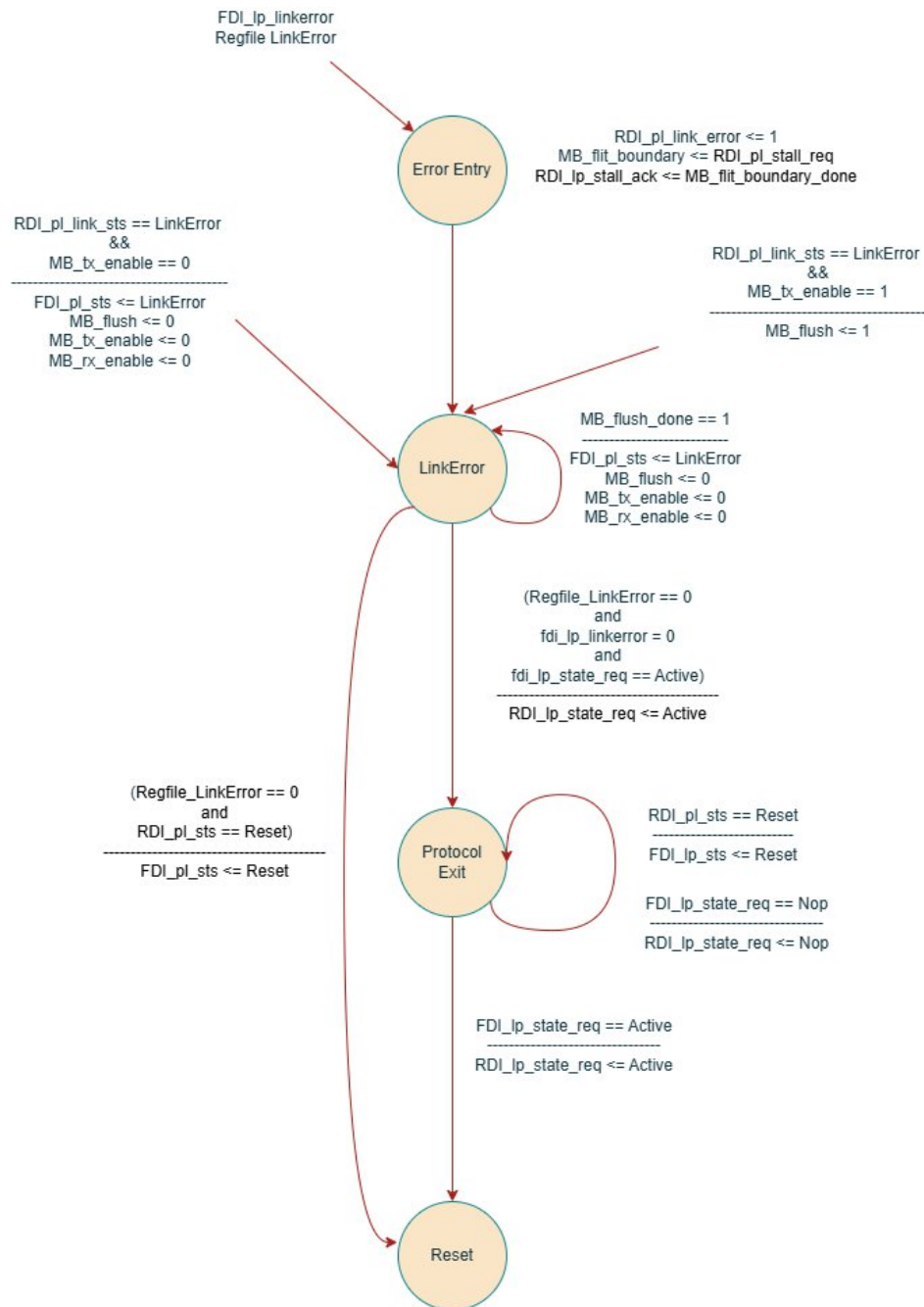
Reset flow completed: The UP Adapter (Die 1) asserts `pl_state_sts = Reset` to the UP Protocol Layer (Die 1). The UP Protocol Layer (Die 1) asserts `pl_state_sts = Reset` to the UP Adapter (Die 1). The DP Protocol Layer (Die 0) asserts `pl_state_sts = Reset` to the DP Adapter (Die 0). The DP Adapter (Die 0) asserts `pl_state_sts = Reset` to the DP Physical layer (Die 0). The DP Physical layer (Die 0) asserts `pl_state_sts = Reset` to the CHANNEL. The CHANNEL asserts `pl_state_sts = Reset` to the UP Physical layer (Die 1), which asserts `pl_state_sts = Reset` to the UP Adapter (Die 1). The UP Adapter (Die 1) then asserts `pl_state_sts = Reset` to the UP Protocol Layer (Die 1).

Start Link Training: The DP Protocol Layer (Die 0) asserts `lp_state_req=NOP->Active` to the DP Adapter (Die 0). The DP Adapter (Die 0) asserts `lp_state_req=NOP->Active` to the DP Physical layer (Die 0). The DP Physical layer (Die 0) asserts `lp_state_req=NOP->Active` to the CHANNEL. The CHANNEL asserts `lp_state_req=NOP->Active` to the UP Physical layer (Die 1), which asserts `lp_state_req=NOP->Active` to the UP Adapter (Die 1). The UP Adapter (Die 1) then asserts `lp_state_req=NOP->Active` to the UP Protocol Layer (Die 1).

Final State: The DP Protocol Layer (Die 0) has `pl_state_sts = Active`. The DP Adapter (Die 0) has `lp_linkerror=0`. The DP Physical layer (Die 0) has `lp_linkerror=0`. The UP Physical layer (Die 1) has `pl_state_sts = Active`. The UP Adapter (Die 1) has `pl_state_sts = Active`. The UP Protocol Layer (Die 1) has `pl_state_sts = Active`.



Our implementation:

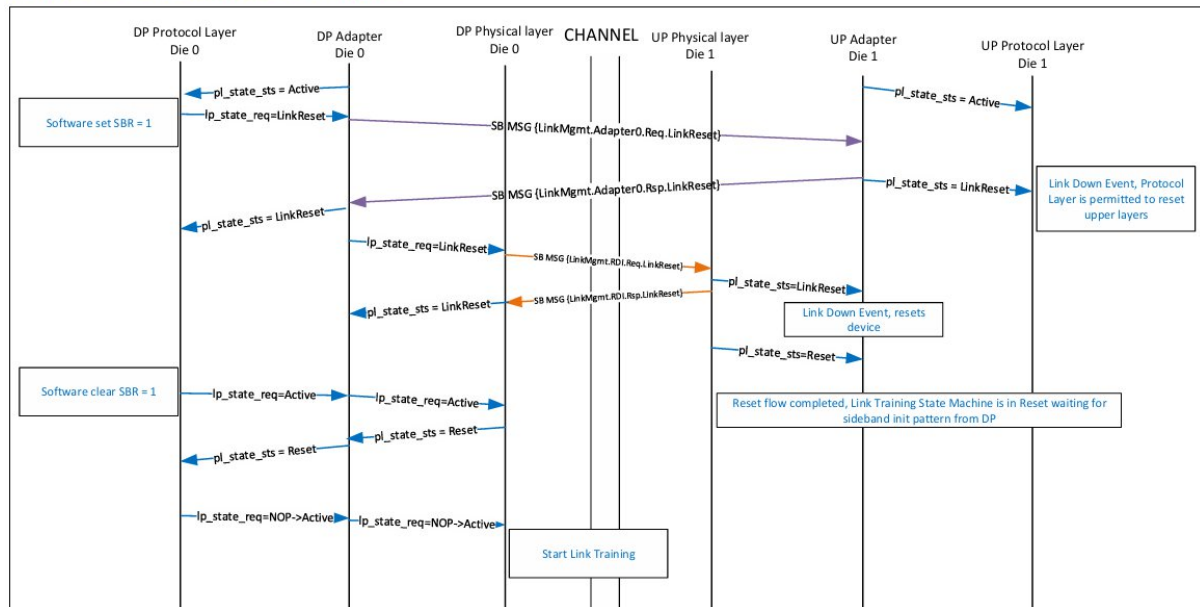


To exit LinkError the Software MUST clear out the Uncorrectable Error Registers in the Register file and the Protocol layer MUST stop requesting LinkError on lp_linkerror

LinkDown:

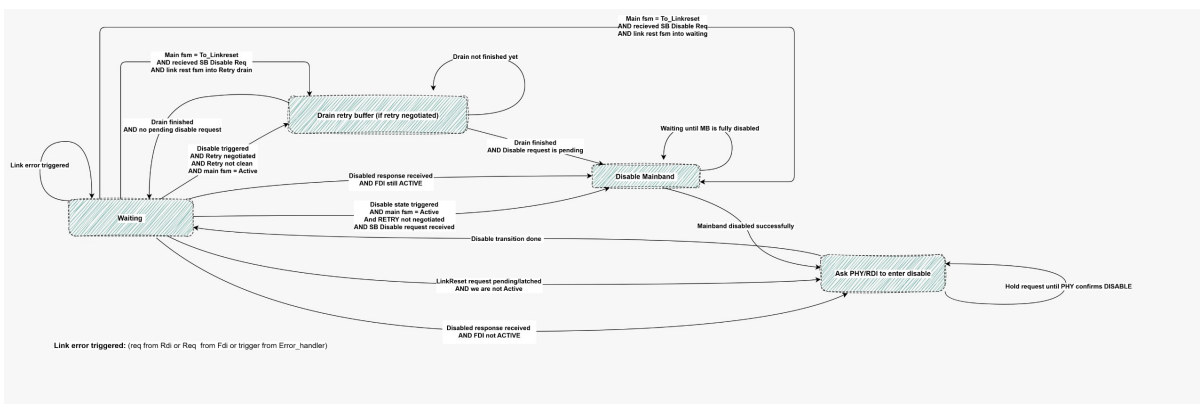
LinkDown state is LinkReset and Disable states. Transition triggers to either state it can be requested from the Protocol, PHY, or internal conditions in the Adapter. The Mainband must try to gracefully drain the flits in the Tx buffer in this case, although should not timeout when entering a down state. If it cannot gracefully drain the buffer it MUST internally clean up the buffer. The ALSM controls the Mainband in this regard, ordering it when to drain and when to internally clean and when to finally disable. Example of LinkDown from the spec:

Figure 10-31. LinkReset Example



Which we implemented as such:

LinkReset



Disable

